

# Ultra Fast Packet Crafter/Ping utility: Blitzping

Anurag Singh   Arjun B Dikshit   Srivaths P   Tanish Yelgoe   Siddharth Rajandekar  
23110035   23110040   23110321   23110328   23110310

## 1 Introduction

Modern high-speed networks demand packet-generation and probing tools capable of operating at hardware line rates with minimal system overhead. Traditional implementations that rely on kernel networking stacks suffer from frequent context switches, buffer copies, and interrupt-driven processing, all of which limit scalability and throughput.

**Blitzping** was originally designed as a packet crafter and network probing utility focused on high performance and low latency. In this project, we extend Blitzping with two major enhancements: (1) a DPDK-based transmission backend for zero-copy, kernel-bypass packet I/O, and (2) a TTL-based traceroute mode with real-time RTT measurement and GUI visualization.

The DPDK integration aims to overcome kernel bottlenecks by leveraging user-space direct access to the NIC through VFIO-PCI drivers and hugepage-backed DMA memory. The traceroute extension, on the other hand, focuses on hop-by-hop path discovery and per-hop latency computation using UDP probes and ICMP feedback. Together, these components transform Blitzping into a modular, high-speed diagnostic tool capable of precise, concurrent packet transmission and network path analysis.

Github Link for our code: [LINK](#)

## 2 Original Blitzping

### 2.1 Overview

The primary goal of Blitzping is to allow packet generation and testing across different network layers, providing low-level access to sockets and protocol headers. The tool operates by constructing custom IP and TCP packets in memory, then sending them through raw sockets or memory-mapped packet buffers.

### 2.2 Program Initialization

The program begins by initializing a ProgramArgs structure, which holds the configuration. The fill\_defaults() function sets default values for key parameters such as:

- Logger verbosity level (LOG\_INFO)
- Number of threads based on available CPU cores
- Default IPv4 header fields (version, TTL, protocol, etc.)

The IPv4 header structure is initialized as follows:

**Listing 1: IPv4 Header Initialization**

```
1 *(program_args->ipv4) = (struct ip_hdr){
2     .ver = 4,
3     .ihl = 5,
4     .ttl = 128,
5     .proto = IP_PROTO_TCP,
6     .len = htons(sizeof(struct ip_hdr) + sizeof(struct tcp_hdr)),
7     .saddr.address = 0,
8     .daddr.address = 0
9 };
```

This sets up a default IP header using the TCP protocol with a total length equal to the sum of the IP and TCP headers. The source and destination addresses are initialized to zero and later filled in at runtime.

### 2.3 Socket Setup

Blitzping utilizes raw POSIX sockets for packet transmission. The function setup\_posix\_socket() creates a socket with type SOCK\_RAW and protocol IPPROTO\_RAW, which allows direct access to the IP layer.

**Listing 2: Raw Socket Creation**

```
1 int socket_descriptor = socket(
2     AF_INET, // IPv4 domain
3     SOCK_RAW, // Raw socket for direct IP access
4     IPPROTO_RAW // Raw IP protocol
5 );
```

The socket is configured to operate in either synchronous or asynchronous mode using the fcntl() system call. In asynchronous mode, the program can send packets without blocking on I/O operations, allowing for high throughput.

For systems supporting Linux AF\_PACKET sockets, Blitzping also includes an alternative setup function, setup\_mmap\_socket(), which uses the kernel's PACKET\_TX\_RING feature. This allows packets to be sent through memory-mapped ring buffers, minimizing system call overhead and improving performance.

### 2.4 Packet Crafting and Transmission Loop

The core functionality of Blitzping resides in the send\_loop() function, which constructs and transmits packets continuously. It maintains an aligned packet buffer that holds the IP and TCP headers.

**Listing 3: Packet Construction**

```
1 _Alignas (_Alignof (max_align_t)) uint8_t
2     packet_buffer[IP_PKT_MTU] = {0};
3
4 struct ip_hdr *ip_header = (struct ip_hdr *)packet_buffer;
5 struct tcp_hdr *tcp_header = (struct tcp_hdr *) (packet_buffer +
6     sizeof (struct ip_hdr));
7
8 *ip_header = *(program_args->ipv4);
9 *tcp_header = (struct tcp_hdr){
10     .sport = htons(rand() % 65536),
11     .dport = htons(80),
12     .seqnum = rand(),
13     .flags.syn = true
14 };
```

This structure initializes a TCP SYN packet with randomized source ports and sequence numbers, effectively simulating connection initiation requests to a specified destination. The IP and TCP headers are placed contiguously in memory for efficient transmission.

The packet is then transmitted using the writev() system call, which allows batching of multiple packets through scatter-gather I/O:

**Listing 4: Packet Transmission Loop**

```

1 for (;;) {
2     next_ip = htonl(program_args->ipv4->saddr.address + (rand() %
3         ip_diff));
4     next_port = htons(rand() % 65536);
5
6     bytes_written = writev(program_args->socket, iov, 37);
7     if (bytes_written == -1) {
8         // Handle EAGAIN / EWOULDBLOCK via poll()
9     }
10    ip_header->saddr.address = next_ip;
11    tcp_header->sport = next_port;
12 }

```

The use of `writev()` and `poll()` ensures efficient asynchronous packet sending and proper socket event handling. The packet buffer is updated in each iteration to randomize source IPs and ports, which can simulate large-scale network scans or stress tests.

## 2.5 Threading and Performance

The `send_packets()` function manages the multithreaded execution of `send_loop()`. Each thread independently crafts and transmits packets, thereby increasing the overall throughput. To reduce memory paging overhead, the program optionally uses `mlockall()` to lock its memory pages, preventing them from being swapped out during high-speed transmission:

**Listing 5: Memory Locking for Performance**

```

1 if (!program_args->advanced.no_mem_lock) {
2     if (mlockall(MCL_FUTURE) == -1) {
3         logger(LOG_ERROR, "Failed to lock memory: %s", strerror(errno));
4     }
5 }

```

## 3 Methodology

### 3.1 Porting Blitzping to DPDK

In Blitzping, we replace the standard socket-based UDP stack with a DPDK-based datapath to achieve kernel-bypass packet I/O. Instead of relying on system calls for send/receive, packets are constructed, managed, and transmitted directly in user space using DPDK's Environment Abstraction Layer (EAL) and poll-mode drivers. This enables zero-copy transmission, NIC offload utilization, and precise control over packet timing.

This subsection documents the implementation details of the UDP datapath that was added to Blitzping and ported to DPDK. It explains the program flow, packet layout, the high-performance send loop that uses DPDK primitives, and the helper functions used for device and memory management.

#### 3.1.1 DPDK mode detection and EAL bootstrap

At program startup the application checks command-line arguments for the presence of a DPDK flag (“`--use-dpdk`”). If present, the program initializes the DPDK Environment Abstraction Layer (EAL) and adjusts `argc/argv` so the rest of the application's argument parser still works:

**Listing 6: DPDK mode detection and EAL initialization (excerpt)**

```

1 bool dpdk_mode = false;
2 for (int i = 1; i < argc; i++) {
3     if (argv[i] && strcmp(argv[i], "--use-dpdk") == 0) {
4         dpdk_mode = true; break;
5     }
6 }
7 if (dpdk_mode) {
8     eal_args_consumed = setup_dpdk_environment(argc, argv);
9     ...
10    socket_descriptor = setup_dpdk_socket(0);
11    ...
12 }

```

- **setup\_dpdk\_environment:** calls `rte_eal_init()`; on success it creates an mbuf pool used for packet buffers. The mbuf pool size and cache parameters are controlled by compile-time macros (e.g. `NUM_MBUEFS`, `MBUF_CACHE_SIZE`). If either EAL init or pool creation fails, the program logs a critical error and aborts.
- After EAL init the program optionally queries and logs the port MAC address using `rte_eth_macaddr_get` and waits for link up with `rte_eth_link_get_nowait`. If the link does not come up within a timeout (30 s in the code), it considers the environment invalid and exits.

#### 3.1.2 DPDK device / socket setup

The function `setup_dpdk_socket(port_id)` performs device configuration and queue setup:

**Listing 7: DPDK socket configuration**

```

1 int setup_dpdk_socket(uint16_t port_id) {
2     struct rte_eth_conf port_conf = {
3         .txmode = {
4             .offloads =
5                 RTE_ETH_TX_OFFLOAD_IPV4_CKSUM |
6                 RTE_ETH_TX_OFFLOAD_UDP_CKSUM,
7         },
8     };
9     const uint16_t nb_rxd = 128;
10    const uint16_t nb_txd = 512;
11    int ret = rte_eth_dev_configure(port_id, 1, 1, &port_conf);
12    rte_eth_rx_queue_setup(port_id, 0, nb_rxd, rte_eth_dev_socket_id(
13        port_id), NULL, mbuf_pool);
14    rte_eth_tx_queue_setup(port_id, 0, nb_txd, rte_eth_dev_socket_id(
15        port_id), NULL);
16
17    ret = rte_eth_dev_start(port_id);
18    rte_eth_dev_set_link_up(port_id);
19    return ret;
20 }

```

- **Offloads:** The transmit path requests IPv4 and UDP checksum offload, so the software can set checksums to zero and rely on NIC hardware to compute them, reducing CPU work and improving throughput.
- **Queues & rings:** RX/TX queue counts and descriptor sizes are configured. The code uses single RX/TX queues per port for simplicity.
- **Mempool usage:** The `mbuf_pool` created during EAL init is passed into RX queue setup. The pool provides the storage for packet payloads (mbufs).

### 3.1.3 Send loop for dpdk

The DPDK send loop continuously randomizes source IP/port fields and transmits packets using a custom transmit wrapper `dpdk_send_packet()`.

**Listing 8: DPDK UDP send loop (excerpt)**

```
1 for (;;) {
2     ip_header->saddr.address = htonl(ntohl(program_args->ipv4->saddr.
3         address) + (rand() % ip_diff));
4     ip_header->ttl = 127;
5     udp_header->sport = htons(rand() % 65536);
6
7     if (dpdk_send_packet(port_id, packet_buffer, packet_length) != 0)
8         { ... }
9
10    uint64_t now_tsc = rte_get_tsc_cycles();
11    if (now_tsc - last_tsc >= one_sec_cycles) {
12        dpdk_monitor(port_id);
13        last_tsc = now_tsc;
14    }
15 }
```

- **Randomization:** The source IP and source port are randomized per-packet as its useful for stress testing.
- **High-resolution timing:** `rte_get_tsc_cycles()` and `rte_get_tsc_hz()` are used to get very precise cycle counters for timing and to trigger periodic monitoring (e.g., per-second statistics).
- **Minimal per-packet work:** Per-packet modifications are limited to a few header fields before transmitting; checksums are offloaded to the NIC to avoid expensive per-packet checksum calculations in software.

#### 3.1.4 DPDK transmit helper

**Listing 9: DPDK transmit implementation (excerpt)**

```
1 int dpdk_send_packet(uint16_t port_id, const void *pkt_data, size_t
2     len) {
3     struct rte_mbuf *m = rte_pktmbuf_alloc(mbuf_pool);
4     void *dst = rte_pktmbuf_append(m, len);
5     memcpy(dst, pkt_data, len);
6
7     m->l2_len = sizeof(struct rte_ether_hdr);
8     m->l3_len = sizeof(struct ip_hdr);
9     m->ol_flags = RTE_MBUF_F_TX_IPV4 | RTE_MBUF_F_TX_IP_CKSUM |
10         RTE_MBUF_F_TX_UDP_CKSUM;
11
12     struct rte_mbuf *bufs[1] = { m };
13     uint64_t start_tsc = rte_get_tsc_cycles();
14     uint16_t sent = rte_eth_tx_burst(port_id, 0, bufs, 1);
15     uint64_t end_tsc = rte_get_tsc_cycles();
16     ...
17     if (sent == 0) { rte_pktmbuf_free(m); return -1; }
18     return (int)sent;
19 }
```

- **mbuf allocation:** A packet mbuf is allocated from the mempool each transmit. The code appends the payload, copies bytes into the mbuf, and sets `pkt_len/data_len` via `rte_pktmbuf_append()`.
- **Transmit API:** `rte_eth_tx_burst()` is used to send the mbuf(s) on the specified port/queue. This API is highly optimized and can send bursts of mbufs with very low overhead.
- **Timing and statistics:** The function measures the send latency using TSC counters and accumulates average send

time for reporting. If TX returns 0 packets sent, the mbuf is freed and an error is returned.

## 3.2 Traceroute Implementation

This subsection describes the traceroute implementation added to Blitzping. The traceroute works by sending UDP probes with incrementing TTL values and listening for ICMP replies (“Time Exceeded” or “Destination Unreachable”) that quote the original probe.

### 3.2.1 Probe construction and transmission

We build minimal UDP probes (IP + UDP headers) in a contiguous, aligned buffer and set the probe source port to a small token that encodes the probe identity (PID, TTL and probe index). For each TTL we transmit a fixed number of probes:

**Listing 10: Send probes for one TTL (core excerpt)**

```
1 for (int p = 0; p < TRACERT_PROBES_PER_TTL; ++p) {
2     uint16_t token = (uint16_t)((getpid() & 0xffff) ^ (ttl << 8) ^ p)
3     ;
4     udp_header->sport = htons(token);
5     udp_header->dport = htons(33434 + ttl);
6     ip_header->ttl = (uint8_t)ttl;
7     clock_gettime(CLOCK_MONOTONIC, &probes[p].ts); // timestamp when
8     sent
9     probes[p].sport = udp_header->sport;
10
11     ssize_t sent = sendto(program_args->socket, packet_buffer,
12         packet_length, 0,
13         (struct sockaddr *)&dest_info, sizeof(
14             dest_info));
15     /* track sent/failed counters */
16 }
```

- The source port encodes a unique token so that the ICMP quote can be mapped back to the original probe.
- Destination UDP port is chosen from the high-range typically used by traceroute (33434 + TTL).
- The send timestamp uses `CLOCK_MONOTONIC` to avoid issues with system clock adjustments.

### 3.2.2 ICMP reception and matching quoted packet

The traceroute opens a raw ICMP socket in non-blocking mode and polls it for replies. Upon receiving an ICMP datagram, it validates the ICMP type (11 = Time Exceeded, 3 = Destination Unreachable), locates the quoted IP header and quoted UDP header inside the ICMP payload, and extracts the quoted UDP source port:

**Listing 11: ICMP receive and quoted-packet parsing**

```
1 ssize_t rlen = recvfrom(icmp_sock, recvbuf, sizeof(recvbuf), 0, (
2     struct sockaddr *)&src_addr, &addrlen);
3 if (rlen >= (ssize_t)sizeof(struct ip_hdr) + 8) {
4     struct ip_hdr *outer_ip = (struct ip_hdr *)recvbuf;
5     int outer_ihl = (outer_ip->ihl & 0x0f) * 4;
6     uint8_t *icmp_ptr = recvbuf + outer_ihl;
7     uint8_t icmp_type = icmp_ptr[0];
8     if (icmp_type != 11 && icmp_type != 3) continue;
9
10    uint8_t *quoted_ptr = icmp_ptr + 8; /* skip ICMP header */
11    struct ip_hdr *quoted_ip = (struct ip_hdr *)quoted_ptr;
12    int q_ihl = (quoted_ip->ihl & 0x0f) * 4;
13    struct udp_hdr *quoted_udp = (struct udp_hdr *)((uint8_t *)
14        quoted_ip + q_ihl);
15 }
```

```

14 uint16_t quoted_sport = quoted_udp->sport;
15 /* match quoted_sport to the outstanding probes */
16 }

```

### 3.2.3 RTT computation

When an ICMP message is matched to a sent probe (via the quoted source port), the code computes RTT as the difference between the current monotonic time and the probe's stored send timestamp:

#### Listing 12: RTT calculation (core excerpt)

```

1 struct timespec now;
2 clock_gettime(CLOCK_MONOTONIC, &now);
3 long ms = (now.tv_sec - probes[i].ts.tv_sec) * 1000L +
4 (now.tv_nsec - probes[i].ts.tv_nsec) / 1000000L;
5 probes[i].answered = true;
6 probes[i].rtt_ms = ms;
7 probes[i].responder = src_addr; /* store responder IP */
8 rs_update(&ttl_stats, ms); /* update running statistics */
9 rtt_array_push(&all_rtts, ms); /* collect for percentiles */

```

Details:

- CLOCK\_MONOTONIC ensures monotonic elapsed times even if the system clock changes.
- RTTs are recorded in milliseconds and aggregated into per-TTL running statistics (min/mean/max/stddev) and a global RTT array for percentile calculations (median, P95).
- The responder's IPv4 address is saved from the src\_addr returned by recvfrom; this identifies the hop that generated the ICMP reply.

### 3.2.4 Timeouts, polling and termination

Probes for each TTL are allowed a timeout (TRACERT\_TIMEOUT\_MS), which is fixed, during which the program polls the ICMP socket. Unanswered probes are marked and reported as such. The traceroute stops early if any probe indicates the destination host (matching the probe's destination address) has been reached:

#### Listing 13: Per-TTL wait and stop condition (core excerpt)

```

1 int remaining_ms = TRACERT_TIMEOUT_MS;
2 while (remaining_ms > 0 && unanswered > 0) {
3     int n = poll(&pfds, 1, remaining_ms);
4     if (n <= 0) break;
5     /* receive and handle ICMPs (see parsing excerpt) */
6     /* recompute remaining_ms using CLOCK_MONOTONIC */
7 }
8 ...
9 if (probes[i].answered && probes[i].responder.sin_addr.s_addr ==
10     dest_info.sin_addr.s_addr) {
11     reached = true;
12 }
13 if (reached) break; /* stop traceroute if destination reached */

```

- (1) **We send UDP probes** with a unique token encoded in the source port and with an incremented TTL field.
- (2) **Routers that expire the TTL** reply with ICMP Time Exceeded messages; the destination may reply with ICMP Destination Unreachable when the probe reaches it.
- (3) **Each ICMP reply contains a quoted portion** of the original probe (at minimum the original IP header and the first 8 bytes of payload). We parse this quoted block to extract the original UDP source port (token).

- (4) **Matching the token** to the outstanding probes lets us identify which probe generated the ICMP reply. The responder IP is taken from the recvfrom() source address and the RTT is computed as the difference between receive time and the probe's stored send timestamp (both using CLOCK\_MONOTONIC).

This design provides accurate per-hop RTT measurements and hop IP identification while being robust to delayed or out-of-order ICMP replies by using explicit per-probe tokens and monotonic timestamps.

## 4 Results and Performance Analysis

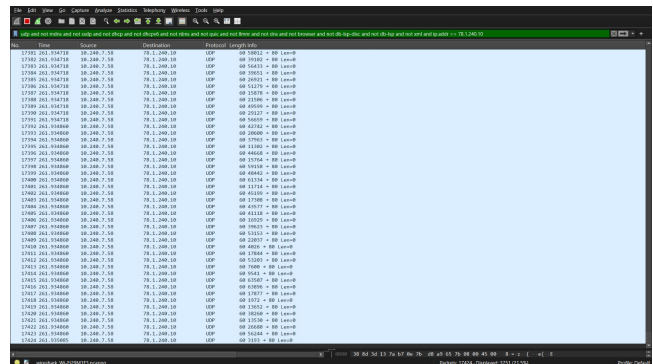
To evaluate the performance impact of DPDK integration in **Blitzping**, we compared the original POSIX-based implementation with the DPDK-accelerated version under both single-threaded and multithreaded configurations. The primary metrics considered were *transmit packets per second (TXpps)*, *transmit bit rate (TXbps)*, and *average latency*. *Note:* Average latency is not reported for DPDK configurations, as it cannot be accurately measured. In DPDK, packets are transmitted directly by the NIC without kernel involvement or completion notifications, making it not possible to determine precise send timestamps for latency calculation.

Additionally, we have implemented the Traceroute functionality with normal Blitzping to determine path taken by the packets to the destination.

**NOTE:** For each configuration, we provide two images, that show the Sender side (GUI) terminal output upon running the program, as well as a screenshot of the Wireshark capture on the Receiving host, providing the proof of the implementation and working of the program

### 4.1 Normal Blitzping (1 Thread)

The baseline Blitzping using standard POSIX sockets achieved an average transmit rate of approximately **105,000 TXpps** and **23.7 Mbps** throughput. The average latency stabilized around **5–6 μs** after initial warm-up. This demonstrates consistent packet generation but highlights the performance ceiling imposed by kernel-level system call overhead.



**Figure 1: Wireshark capture of original Blitzping with 1 thread.**

| Configuration       | TXpps (avg) | TXbps (avg) | Avg. Latency  | Observation                                |
|---------------------|-------------|-------------|---------------|--------------------------------------------|
| Normal (1 thread)   | ~105k       | ~23.7 Mbps  | 5–6 $\mu$ s   | Stable baseline; kernel-limited            |
| Normal (20 threads) | ~5.8k       | ~1.2 Mbps   | 19–25 $\mu$ s | Degraded due to contention                 |
| DPDK (1 thread)     | ~472k       | ~159 Mbps   | –             | 4.5 $\times$ gain; efficient kernel bypass |
| DPDK (20 threads)   | ~14k        | ~4.8 Mbps   | –             | Improper scaling; queue conflicts          |

Table 1: Comparison of Blitzping performance across configurations.

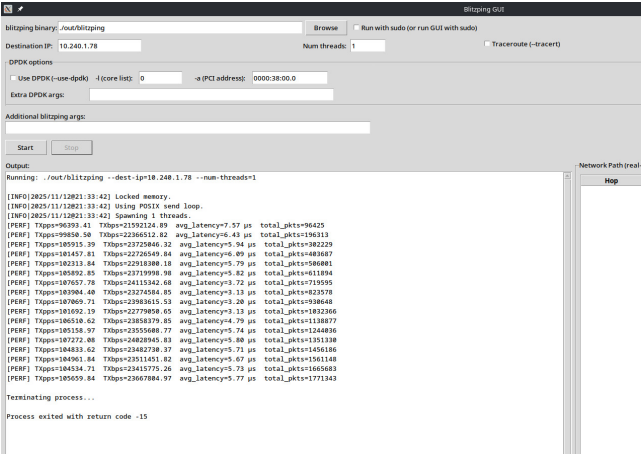


Figure 2: Terminal output of original Blitzping with 1 thread.

4.2 Normal Blitzping (20 Threads)

When scaling to 20 threads, throughput did not increase as expected. Instead, it dropped to an average of 5,500–6,000 TXpps and around 1.2 Mbps, while average latency increased to 19–25  $\mu$ s. The performance decline is attributed to kernel contention, context switching, and synchronization overhead among threads sharing the same kernel networking resources.

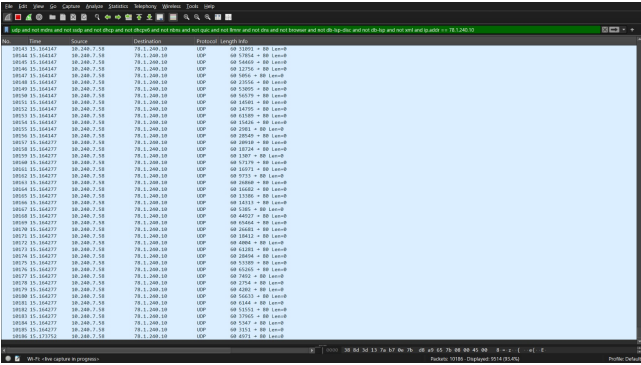


Figure 3: Wireshark capture of original Blitzping with 20 threads.

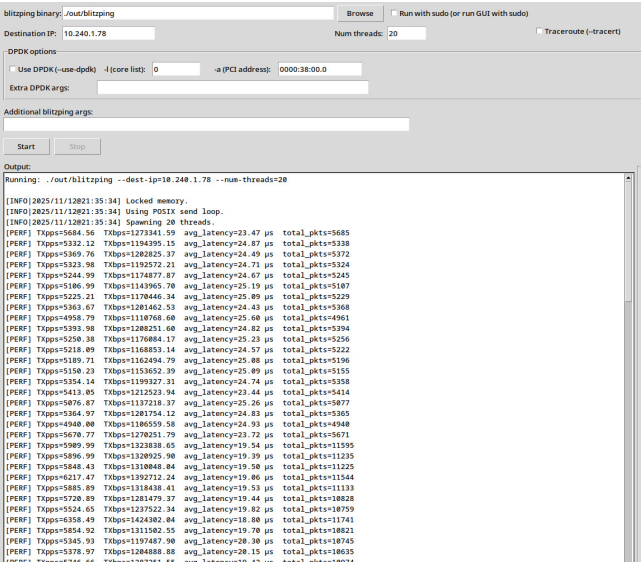


Figure 4: Terminal output of original Blitzping with 20 threads.

4.3 DPDK Blitzping (1 Thread)

In contrast, the DPDK-enabled version of Blitzping exhibited a dramatic performance improvement. The single-threaded DPDK run achieved an average of approximately 472,000 TXpps and 159 Mbps throughput, representing a 4.5 $\times$  increase compared to the POSIX version. This improvement arises from DPDK’s zero-copy, user-space packet I/O path, which eliminates kernel and system call overhead.

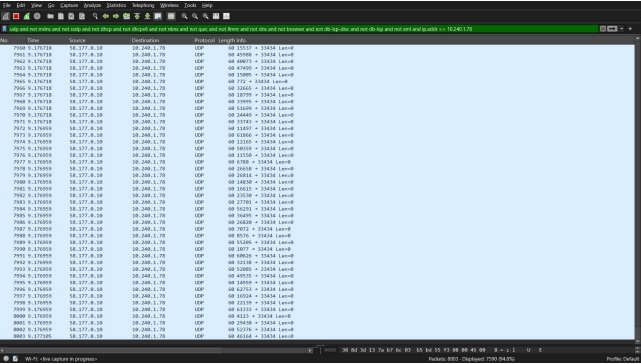


Figure 5: Wireshark capture of DPDK output with 1 thread.

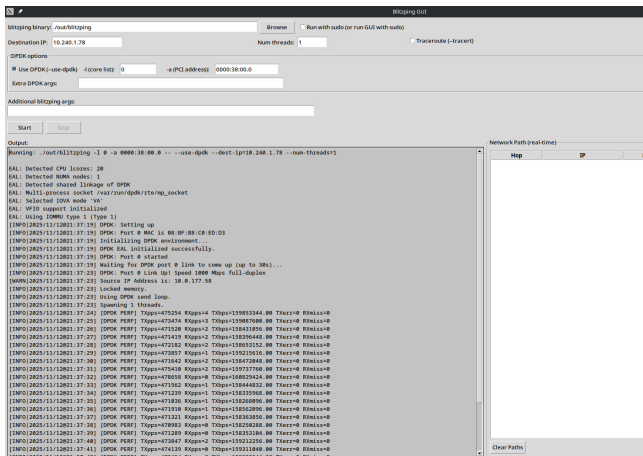


Figure 6: Terminal output of DPDK with 1 threads.

## 4.4 DPDK Blitzping (20 Threads)

The multithreaded DPDK configuration showed suboptimal behavior, producing about **14,000 TXpps**. This likely results from improper thread-to-queue mapping or contention within a single transmit queue of NIC, emphasizing the need for correct queue allocation and NUMA-aware binding to scale performance effectively. However it still exhibits a **4x** increase over the bandwidth demonstrated by Normal Blitzping, emphasizing the utility of hardware driven acceleration provided by DPDK.

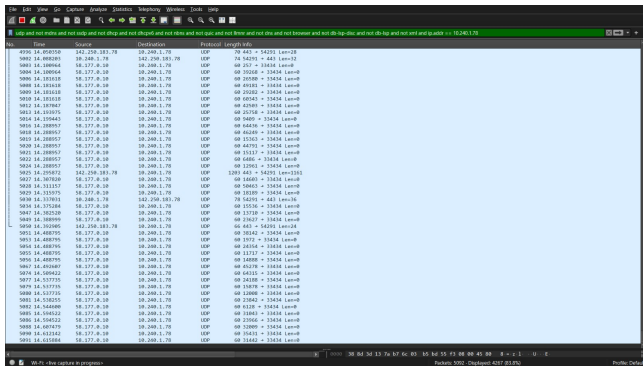


Figure 7: Wireshark capture of DPDK output with 20 threads.

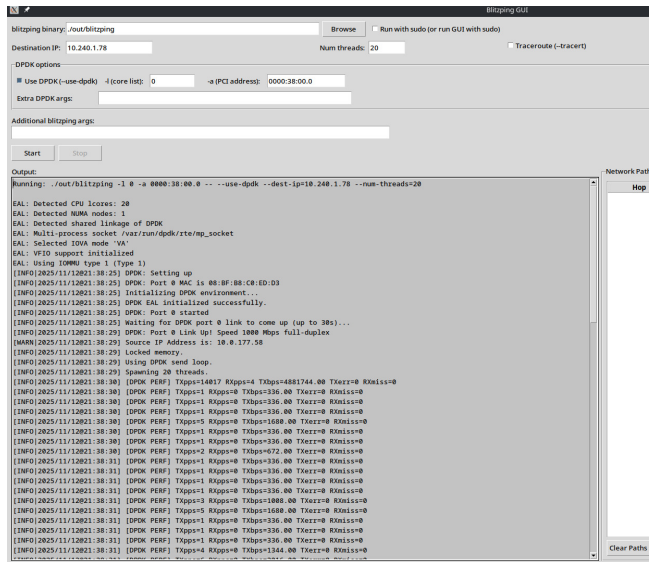


Figure 8: Terminal output of DPDK with 20 threadss.

## 4.5 Key Takeaways

- DPDK integration delivers a substantial performance gain in single-threaded mode.
- POSIX-based multithreading is inefficient due to kernel contention and overheads due to context switching.
- DPDK’s multithreaded performance also drops in comparison to single threaded version but still exhibits a substantial performance gain over the POSIX-multithreaded approach.
- Overall, DPDK enables order-of-magnitude improvement in throughput and transmission rate for high-performance probing.

## 4.6 Why DPDK Improved Throughput

DPDK achieves higher throughput primarily by bypassing the traditional kernel networking stack, thereby eliminating costly system calls, context switches, and `sk_buff` allocations. By operating entirely in user space with preallocated `rte_mbuf` memory pools, DPDK enables zero-copy packet handling and minimizes dynamic memory operations. This direct, streamlined data path allows Blitzping to sustain significantly higher packet transmission rates and achieve near line-rate performance even under heavy load.

## 4.7 Traceroute Functionality and Results

The traceroute extension of Blitzping was evaluated to assess hop-by-hop latency measurement and path visualization toward Google’s public DNS server (8.8.8.8). The tool successfully identified **12 hops** from the local gateway to the destination, with each hop’s average round-trip time (RTT) displayed dynamically in the GUI. The discovered route is shown in the below table

Table 2: Traceroute summary from Blitzping to 8.8.8.8

| Hop | IP Address     | Avg RTT (ms) |
|-----|----------------|--------------|
| 1   | 10.240.0.2     | 1.2          |
| 2   | 10.3.0.29      | 1.6          |
| 3   | 10.3.0.5       | 3.6          |
| 4   | 172.16.4.7     | 3.1          |
| 5   | 14.139.98.1    | 9.5          |
| 6   | 10.117.81.253  | 2.8          |
| 7   | 10.154.8.137   | 12.6         |
| 8   | 10.255.239.170 | 11.2         |
| 9   | 10.152.7.214   | 17.4         |
| 10  | 72.14.204.62   | 12.1         |
| 11  | 8.8.8.8        | 15.3         |

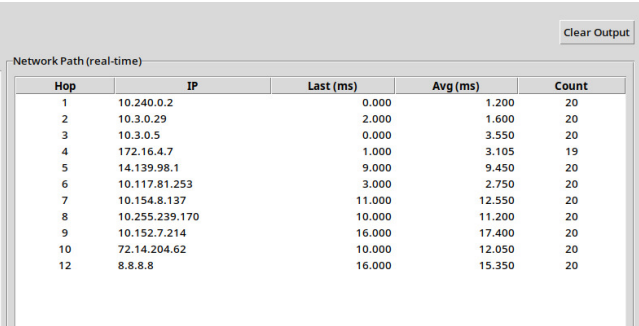


Figure 11: Traceroute path example (Multi Threaded).

Average hop latencies ranged from **1.2 ms** at the local gateway to **15.3 ms** at the destination, showing a progressive RTT increase across the network path. The traceroute summary indicated stable performance and low jitter, confirming the precision of per-hop RTT computation.

Overall, the traceroute module combines high-speed packet generation with accurate hop-by-hop timing, outperforming traditional utilities by achieving faster probe cycles and fine-grained latency insight through concurrent probing and optimized event handling.

5 Conclusion

In this project, we successfully extended Blitzping, a lightweight, high-performance network packet crafting and probing tool, by integrating DPDK for user-space packet transmission and adding a TTL-based traceroute feature for network path discovery.

The transition from the traditional socket-based implementation to a DPDK-based design allowed Blitzping to bypass the Linux kernel’s networking stack and interact directly with the NIC hardware through DMA and poll-mode drivers. This architectural shift removed costly context switches and system calls, resulting in a substantial performance improvement: on our test hardware, throughput increased by roughly **4x** with DPDK enabled.

The traceroute component enhanced the network visibility of Blitzping. It enabled hop-by-hop latency measurement using UDP probes and ICMP replies, providing an end-to-end view of the network path while maintaining RTT calculations. Together, these features make Blitzping both a performance-oriented packet generator and a practical diagnostic tool.

Beyond the quantitative results, this project demonstrated the trade-offs between kernel and user-space networking approaches:

- Kernel sockets are easier to use and naturally support features like ICMP reception but introduce latency and syscall overhead.
- DPDK offers massive throughput gains and precise control but requires explicit management of memory, buffers, and hardware configuration.

The experience also highlighted how low-level optimizations, such as using hugepages, busy-poll loops, and NIC offloads, can drastically change the performance profile of network applications even on commodity hardware.

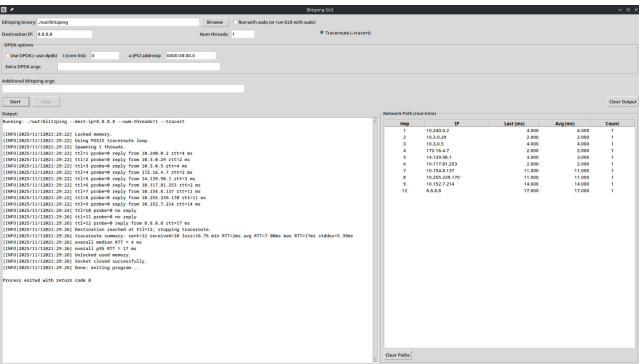


Figure 9: Traceroute output.

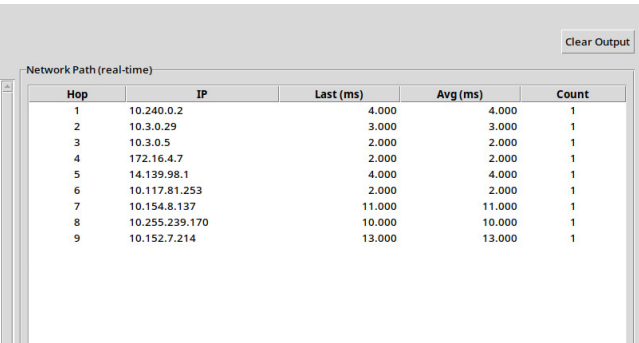


Figure 10: Traceroute path (Single threaded).

## Future Work

There are several directions in which this work can be extended:

- **Multi-core scaling:** Extend the DPDK send loop to use multiple transmit (TX) and receive (RX) queues with dedicated CPU cores. This will allow near-line-rate throughput on multi-gigabit interfaces.

In conclusion, this project showed that combining the speed of DPDK with the flexibility of Blitzping can bridge the gap between traditional network utilities and high-performance packet processing frameworks. It serves as a foundation for future work in network measurement tools that are both efficient and hardware-aware, capable of operating at modern data-center speeds without sacrificing transparency or control.

## References

- [1] CAI, Q., CHAUDHARY, S., VUPPALAPATI, M., HWANG, J., AND AGARWAL, R. Understanding host network stack overheads. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (New York, NY, USA, 2021), SIGCOMM '21, Association for Computing Machinery, p. 65–77.
- [2] DPDK PROJECT. Installing and setting up dpdk on linux, 2023. YouTube video, Accessed: 2025-10-15.
- [3] DPDK PROJECT. Introduction to dpdk - high performance packet processing, 2023. YouTube video, Accessed: 2025-10-15.
- [4] DPDK PROJECT. Dpdk api documentation, 2025. Accessed: 2025-10-15.
- [5] DPDK PROJECT. Linux drivers guide - dpdk 25.07 documentation, 2025. Accessed: 2025-10-15.
- [6] STEPHAN, A., AND WÜSTRICH, L. The path of a packet through the linux kernel. *Technical University of Munich, Chair of Network Architectures and Services, School of Computation, Information and Technology* (2024).
- [7] THRAETAONA / FEREYDOUN MEMARZANJANY. Blitzping: A very high-speed, configurable, and portable packet-crafting utility. <https://github.com/Thraetaona/Blitzping>, 2024. Accessed: 2025-10-15, open source repository under GPL-3.0.